

Project Assignment: Expense Tracker (Console-Based with File I/O)

Task

Design and develop a **console-based Expense Tracker application** using Core Java that allows users to manage daily expenses with persistent storage using a **CSV file**.

Develop a Java application that enables users to:

- Record daily expenses with details
 - Categorize expenses
 - View and filter expenses
 - Calculate total spending
 - Persist data using file handling (CSV format)
-

Technology Stack

- Core Java
 - Java Collections (`ArrayList`)
 - File I/O (`FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`)
 - Exception Handling
-

Project Structure (Standard Package Design)

```
com.expensetracker
├── model
│   └── Expense.java
├── service
│   └── ExpenseService.java
├── util
│   └── FileUtil.java
└── main
    └── ExpenseTrackerApp.java
```

\



Class Design & Responsibilities

1 Expense.java (Model Class)

Package: `com.expensetracker.model`

◆ Variables

- `String` description
- `double` amount
- `String` category

◆ Constructors

- Parameterized constructor

◆ Methods

- `String` toString()
 - `String` toCSV() → Convert object to CSV format
 - `static Expense` parseExpense(`String` line) → Convert CSV line to object
 - Getters:
 - `getDescription()`
 - `getAmount()`
 - `getCategory()`
-

2 ExpenseService.java (Business Logic Layer)

Package: `com.expensetracker.service`

◆ Variables

- `ArrayList<Expense>` expenses

◆ Methods

- `void` addExpense(`Expense` expense)
 - `List<Expense>` getAllExpenses()
 - `List<Expense>` getExpensesByCategory(`String` category)
 - `double` calculateTotalExpenses()
 - `void` loadExpenses()
 - `void` saveExpenses()
-

3 FileUtil.java (Utility Class for File Handling)

Package: `com.expensetracker.util`

◆ Constants

- `String FILE_NAME = "expenses.csv"`

◆ Methods

- `List<Expense> readFromFile()`
- `void writeToFile(List<Expense> expenses)`

◆ Exceptions Handled

- `FileNotFoundException`
 - `IOException`
 - `IllegalArgumentException`
-

4 ExpenseTrackerApp.java (Main Class / UI Layer)

Package: `com.expensetracker.main`

◆ Responsibilities

- User interaction via console
- Menu handling
- Calling service methods

◆ Methods

- `main(String[] args)`
 - `void showMenu()`
 - `void handleUserChoice(int choice)`
-



Functional Requirements

✓ 1. Add Expense

- Input:
 - Description
 - Amount
 - Category
 - Store in memory + save to file
-

✓ 2. View All Expenses

- Display all stored expenses
-

✓ 3. View Expenses by Category

- Filter based on user input
 - Case-insensitive comparison
-

✓ 4. Calculate Total Expenses

- Sum all expense amounts
-

✓ 5. Persistent Storage

- Load data from CSV at startup
 - Save data after every modification
-



CSV File Format

description,amount,category

Lunch,150,Food

Bus Ticket,50,Travel

Movie,300,Entertainment

Exception Handling Requirements

Handle the following exceptions

- Invalid numeric input (`InputMismatchException`)
 - File not found (`FileNotFoundException`)
 - General I/O errors (`IOException`)
 - Invalid CSV format (`IllegalArgumentException`)
-

Validation Rules

- Amount must be positive
 - Description & category should not be empty
 - Handle invalid menu choices
-

Sample Execution Output:

Expense Tracker Menu:

1. Add Expense
2. View All Expenses
3. View Expenses by Category
4. Calculate Total Expenses
5. Exit

Enter your choice: 1

Enter description: Lunch

Enter amount: 150

Enter category: Food

Expense added.

Enter your choice: 1

Enter description: Bus Ticket

Enter amount: 50

Enter category: Travel

Expense added.

Enter your choice: 2

All Expenses:

Lunch - 150.0 (Food)

Bus Ticket - 50.0 (Travel)

Enter your choice: 3
Enter category to filter by: Food

Expenses in category 'Food':
Lunch - 150.0 (Food)

Enter your choice: 4

Total Expenses: 200.0

Enter your choice: 5

Goodbye!

Next:

- Add date field to expenses
- Sort expenses by amount/date
- Delete or update expense